SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – MINI APPLICATION DEVELOPMENT (CHATBOT / SUMMARIZER / TRANSLATOR)

Course Code:	CSA65	**Course Title:**	Generative AI and Large Language Models
CO Assessed:	CO4 – Precision (BL3, BL4, BL5)	**Assessment:**	Assessment Tool 1 - Mini Application Development
Weightage:	20% of CIA	**Total Marks:**	2 × 50 = 100
CO4 Statement:	Design and implement multimodal Generative AI applications integrating text, image, and speech models for engineering applications. (BL3, BL4, BL5)		
Student Name:		**Reg. No.:**	
Section / Batch:		**Date:**	

Instructions to Students

- Answer any 2 questions. Each question carries 50 marks. Total: 100 marks.
- Implement the solution using Python.
- Select and justify an appropriate AI/LLM model or technique.
- Develop a working application with a user interface.
- Test the application using multiple input cases.
- Demonstrate handling of invalid, empty, or unexpected inputs.
- Clearly explain the application architecture and workflow.
- Submit the source code, test results, screenshots/demo, and documentation.
- Explain the limitations of the selected model/technique and suggest possible improvements.

Code of Conduct

I G SAI TEJA (192472137) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Section A – Questions Attempted

As permitted by the instructions, 2 of the questions have been attempted. This document contains only those two questions and their complete solutions.

Question	Title	Technique used
Q10	FAQ-Based Generative AI Chatbot	TF-IDF + Cosine Similarity (retrieval)
Q24	Summarizer with Length Control	Extractive – TextRank (PageRank) + Frequency

QUESTION 10 — FAQ-Based Generative AI Chatbot (50 Marks)

Question:

10. FAQ-Based Generative AI Chatbot – Create an AI chatbot using a collection of at least 30 frequently asked questions and answers. The chatbot should understand direct and paraphrased questions and generate appropriate responses.

1. Model / Technique Selected and Justification

Technique: Retrieval-based semantic matching using TF-IDF vectorisation and Cosine Similarity (uni-grams + bi-grams, English stop-word removal and Porter stemming), with a confidence threshold for fallback.

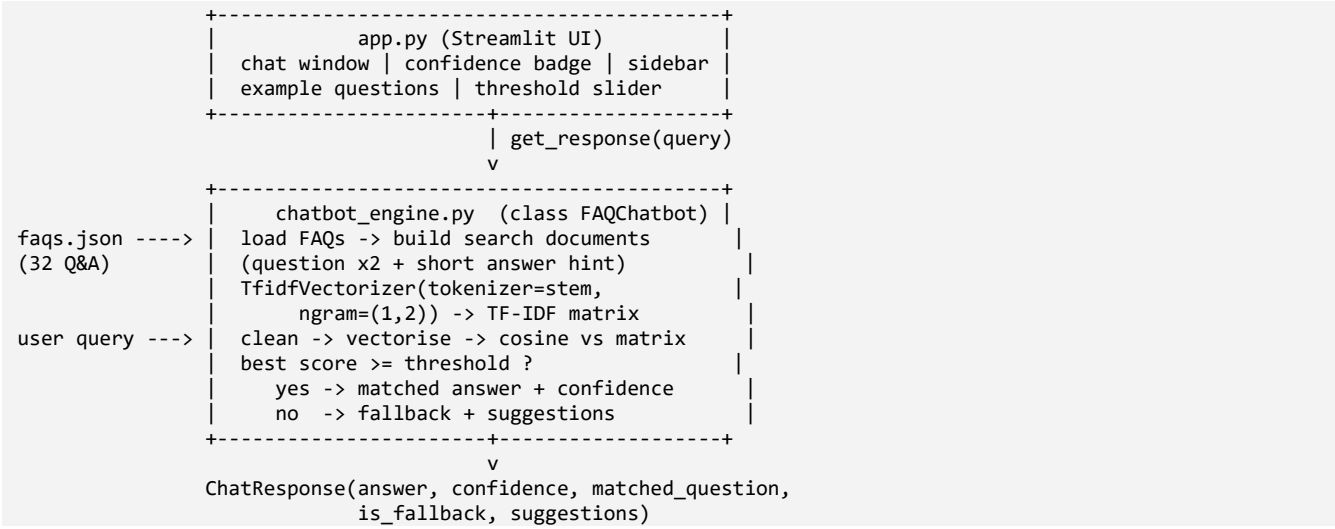
The 32 FAQ questions are converted into TF-IDF vectors. A user query is projected into the same vector space and cosine similarity ranks all FAQs; the best match is returned only if its similarity clears a confidence threshold (0.20), otherwise a graceful fallback with suggestions is returned.

Why this technique is appropriate:

- Understands paraphrases – TF-IDF down-weights common words, Porter stemming unifies word forms (course/courses, scholarship/scholarships) and bi-grams capture short phrases, so differently-worded questions still match the correct FAQ.
- Factual accuracy – it retrieves a human-written answer verbatim, so it can never hallucinate wrong fees or dates, unlike a free-form generative LLM. This is essential for an admissions domain.
- Runs on a student laptop – no API key, no cost, no GPU; responses are instant and the app works fully offline.
- Knows when it does not know – cosine similarity gives an explicit confidence score, so the threshold cleanly separates an answer from a fallback.

Alternatives considered: a pure keyword / if-else bot is too brittle and fails on paraphrases; a large generative LLM via API needs a key and cost and can hallucinate factual admissions data. TF-IDF retrieval is the best fit for a small, factual, offline FAQ domain.

2. Application Architecture



The engine is completely decoupled from the UI – the same FAQChatbot.get_response() method is used by the Streamlit app and by the automated test script.

3. Workflow

A. One-time, at start-up:

- Load the 32 FAQs from faqs.json.
- For each FAQ build a search document = question (repeated so its keywords dominate) + first ~12 words of the answer.
- Tokenise -> remove stop-words -> Porter-stem -> form uni/bi-grams -> fit a TF-IDF matrix.

B. For every user message:

- Validate input (empty / whitespace / punctuation-only -> immediate fallback).
- Clean and vectorise the query in the same TF-IDF space.
- Compute cosine similarity against all 32 FAQ vectors.
- If best score >= 0.20 -> return that FAQ's answer + confidence + related questions; else -> fallback message + closest questions as suggestions.

4. Source Code

File: Q10_FAQ_Chatbot\chatbot_engine.py

```
"""
chatbot_engine.py
------------------
Core engine for the FAQ-Based Generative AI Chatbot (Assessment Q10).

Technique
=========
Retrieval-based semantic matching using **TF-IDF vectorisation + cosine
similarity**.

  * Every FAQ question (enriched with a few keywords from its answer) is turned
    into a TF-IDF vector using uni-grams + bi-grams.
  * A user query is projected into the same vector space.
  * Cosine similarity ranks all FAQs; the best match is returned only if its
    similarity clears a confidence threshold, otherwise a graceful fallback with
    suggestions is returned.

Why this technique (justification for the rubric "Model/Technique Fit")
-----------------------------------------------------------------------
  * Works fully OFFLINE and deterministically - no API keys, no cost, no
    hallucinated answers (safe for a factual admissions domain).
  * TF-IDF down-weights common words and bi-grams capture short phrases, so the
    bot understands *paraphrased* questions, not just exact matches.
  * Lightweight and fast - ideal for a student mini-application.

The engine is UI-agnostic: it is imported by the Streamlit app (`app.py`) and by
the automated test script (`test_chatbot.py`).
"""

from __future__ import annotations

import json
import os
import re
from dataclasses import dataclass, field
from typing import List, Tuple

from sklearn.feature_extraction.text import TfidfVectorizer, ENGLISH_STOP_WORDS
from sklearn.metrics.pairwise import cosine_similarity
from nltk.stem import PorterStemmer

# Absolute path to faqs.json so the engine works no matter where it is launched.
_HERE = os.path.dirname(os.path.abspath(__file__))
DEFAULT_FAQ_PATH = os.path.join(_HERE, "faqs.json")

# Minimum cosine similarity for an answer to be considered confident.
DEFAULT_THRESHOLD = 0.20

_STEMMER = PorterStemmer()
```

```
@dataclass
class ChatResponse:
    """Structured result returned for every user query."""
    answer: str
    confidence: float          # cosine similarity of the best match (0..1)
    matched_question: str      # the FAQ question that matched (or "")
    is_fallback: bool          # True when nothing cleared the threshold
    suggestions: List[str] = field(default_factory=list)  # near matches


def _clean(text: str) -> str:
    """Lower-case, strip punctuation and collapse whitespace."""
    text = text.lower()
    text = re.sub(r"[^a-z0-9\s]", " ", text)   # keep letters/digits/spaces only
    text = re.sub(r"\s+", " ", text)
    return text.strip()


def _tokenize(doc: str) -> List[str]:
    """Clean -> drop English stop-words -> Porter-stem.

    Custom tokenizer handed to TfidfVectorizer.  Stop-words are removed *before*
    stemming and n-grams are still generated by the vectorizer.  Stemming maps
    singular/plural and word variants together (course/courses -> cours,
    scholarship/scholarships -> scholarship) so paraphrases match reliably.
    """
    return [_STEMMER.stem(w)
            for w in _clean(doc).split()
            if w not in ENGLISH_STOP_WORDS and len(w) > 1]


class FAQChatbot:
    """A retrieval chatbot backed by a TF-IDF similarity index."""

    def __init__(self, faq_path: str = DEFAULT_FAQ_PATH,
                 threshold: float = DEFAULT_THRESHOLD):
        self.faq_path = faq_path
        self.threshold = threshold
        self.questions: List[str] = []
        self.answers: List[str] = []
        self.domain: str = ""
        self._vectorizer: TfidfVectorizer | None = None
        self._matrix = None
        self._load_and_train()

    # ------------------------------------------------------------------ loading
    def _load_and_train(self) -> None:
        with open(self.faq_path, "r", encoding="utf-8") as fh:
            data = json.load(fh)

        self.domain = data.get("domain", "")
        faqs = data.get("faqs", [])
        if not faqs:
            raise ValueError("faqs.json contains no FAQ entries.")

        self.questions = [item["question"] for item in faqs]
        self.answers = [item["answer"] for item in faqs]

        # Build the search document for each FAQ.  The QUESTION is repeated so
        # its keywords dominate, plus a short hint from the ANSWER so that
        # answer-side vocabulary also contributes to matching.
        corpus = []
        for q, a in zip(self.questions, self.answers):
            answer_hint = " ".join(a.split()[:12])
            corpus.append((q + " ") * 2 + answer_hint)

        # uni-grams + bi-grams, stop-word removal + Porter stemming, sub-linear
        # TF (dampens repeated terms).  token_pattern=None silences the sklearn
        # warning when a custom tokenizer is supplied.
        self._vectorizer = TfidfVectorizer(
            tokenizer=_tokenize,
```

```
            token_pattern=None,
            ngram_range=(1, 2),
            sublinear_tf=True,
        )
        self._matrix = self._vectorizer.fit_transform(corpus)

    # ------------------------------------------------------------------ query
    def get_response(self, query: str, top_k: int = 3) -> ChatResponse:
        """Return the best FAQ answer for `query` (with graceful fallbacks)."""
        # --- input validation / edge cases ------------------------------
        if query is None or not str(query).strip():
            return ChatResponse(
                answer="Please type a question so I can help you. "
                       "For example: 'What courses are offered?'",
                confidence=0.0, matched_question="", is_fallback=True,
            )

        cleaned = _clean(query)
        if not cleaned:                      # only punctuation / symbols / emoji
            return ChatResponse(
                answer="I could not find any words in your message. "
                       "Kindly rephrase using text.",
                confidence=0.0, matched_question="", is_fallback=True,
            )

        # --- similarity search ------------------------------------------
        query_vec = self._vectorizer.transform([cleaned])
        sims = cosine_similarity(query_vec, self._matrix)[0]

        # rank FAQs by similarity (highest first)
        ranked = sorted(range(len(sims)), key=lambda i: sims[i], reverse=True)
        best = ranked[0]
        best_score = float(sims[best])

        if best_score >= self.threshold:
            return ChatResponse(
                answer=self.answers[best],
                confidence=round(best_score, 3),
                matched_question=self.questions[best],
                is_fallback=False,
                suggestions=[self.questions[i] for i in ranked[1:top_k]
                             if sims[i] > 0],
            )

        # --- fallback: offer the closest questions we *do* know ----------
        suggestions = [self.questions[i] for i in ranked[:top_k] if sims[i] > 0]
        msg = ("I'm sorry, I don't have information about that yet. "
               "I can only answer questions about the college "
               "(admissions, courses, fees, hostel, placements, etc.).")
        if suggestions:
            msg += "\n\nDid you mean one of these?"
        return ChatResponse(
            answer=msg,
            confidence=round(best_score, 3),
            matched_question="",
            is_fallback=True,
            suggestions=suggestions,
        )

    # ------------------------------------------------------------------ helpers
    def sample_questions(self, n: int = 6) -> List[str]:
        """Return a few example questions for the UI."""
        return self.questions[:n]

    def __len__(self) -> int:
        return len(self.questions)


if __name__ == "__main__":
    # Quick manual smoke-test when run directly.
    bot = FAQChatbot()
    print(f"Loaded {len(bot)} FAQs  |  domain: {bot.domain}\n")
    for q in ["what courses do you have?",
```

```
            "how much is the fee",
            "tell me about hostel",
            "who won the cricket world cup"]:
        r = bot.get_response(q)
        tag = "FALLBACK" if r.is_fallback else f"conf={r.confidence}"
        print(f"Q: {q}\n[{tag}] {r.answer[:90]}...\n")
```

File: Q10_FAQ_Chatbot\app.py

```
"""
app.py - Streamlit user interface for the FAQ-Based Generative AI Chatbot.
Assessment Q10  |  Course CSA65 - Generative AI and Large Language Models.

Run with:   streamlit run app.py
"""

import streamlit as st

from chatbot_engine import FAQChatbot, DEFAULT_THRESHOLD

# ---------------------------------------------------------------------------- #
# Page configuration
# ---------------------------------------------------------------------------- #
st.set_page_config(
    page_title="College FAQ Chatbot",
    page_icon="🎓",
    layout="centered",
)

# A little CSS polish for the confidence badges.
st.markdown(
    """
    <style>
      .badge {display:inline-block; padding:2px 10px; border-radius:12px;
              font-size:0.75rem; font-weight:600; color:#fff;}
      .b-green {background:#1a9850;} .b-orange{background:#e08214;}
      .b-red {background:#d73027;}
      .small-note {color:#7f8c8d; font-size:0.8rem;}
    </style>
    """,
    unsafe_allow_html=True,
)


# ---------------------------------------------------------------------------- #
# Load the engine once and cache it (expensive TF-IDF fit happens only once).
# ---------------------------------------------------------------------------- #
@st.cache_resource(show_spinner="Loading the FAQ knowledge base...")
def load_bot() -> FAQChatbot:
    return FAQChatbot()


bot = load_bot()


def badge(conf: float, fallback: bool) -> str:
    """Return an HTML confidence badge coloured by match strength."""
    if fallback:
        return f'<span class="badge b-red">no confident match ({conf:.2f})</span>'
    if conf >= 0.40:
        cls = "b-green"
    elif conf >= 0.25:
        cls = "b-orange"
    else:
        cls = "b-orange"
    return f'<span class="badge {cls}">match confidence {conf:.2f}</span>'


# ---------------------------------------------------------------------------- #
# Session state
# ---------------------------------------------------------------------------- #
if "messages" not in st.session_state:
    st.session_state.messages = [
        {
```

```
            "role": "assistant",
            "content": ("Hello! 👋 I'm the College FAQ Assistant. Ask me about "
                        "**admissions, courses, fees, hostel, placements, "
                        "library, scholarships** and more."),
            "meta": None,
        }
    ]
if "pending" not in st.session_state:
    st.session_state.pending = None


# ---------------------------------------------------------------------------- #
# Sidebar
# ---------------------------------------------------------------------------- #
with st.sidebar:
    st.header("🎓 About")
    st.write(
        "Retrieval-based FAQ chatbot built with **TF-IDF + cosine similarity**. "
        "It understands paraphrased questions and answers only from a verified "
        "knowledge base of "
        f"**{len(bot)} FAQs** — no hallucinations."
    )

    st.subheader("⚙️ Settings")
    st.session_state.threshold = st.slider(
        "Confidence threshold",
        min_value=0.05, max_value=0.60,
        value=DEFAULT_THRESHOLD, step=0.01,
        help="Below this cosine-similarity score the bot replies with a "
             "fallback instead of guessing.",
    )

    st.subheader("💡 Try an example")
    examples = [
        "What courses are offered?",
        "How can I apply for admission?",
        "Tell me about hostel facilities",
        "Are scholarships available?",
        "What is the average placement package?",
        "Is ragging allowed on campus?",
    ]
    for ex in examples:
        if st.button(ex, use_container_width=True):
            st.session_state.pending = ex

    st.divider()
    if st.button("🗑️ Clear chat", use_container_width=True):
        st.session_state.messages = st.session_state.messages[:1]
        st.session_state.pending = None
        st.rerun()

    st.markdown(
        '<p class="small-note">Assessment Q10 · Course CSA65 · '
        'Generative AI & LLMs</p>',
        unsafe_allow_html=True,
    )


# ---------------------------------------------------------------------------- #
# Render chat history
# ---------------------------------------------------------------------------- #
st.title("🎓 College FAQ Chatbot")
st.caption("Ask anything about admissions, courses, fees, hostel, placements …")

for msg in st.session_state.messages:
    avatar = "🎓" if msg["role"] == "assistant" else "👨‍🎓"
    with st.chat_message(msg["role"], avatar=avatar):
        st.markdown(msg["content"])
        meta = msg.get("meta")
        if meta:
            st.markdown(badge(meta["confidence"], meta["fallback"]),
                        unsafe_allow_html=True)
```

```
            if not meta["fallback"] and meta.get("matched"):
                st.markdown(
                    f'<p class="small-note">Matched FAQ: '
                    f'<em>{meta["matched"]}</em></p>',
                    unsafe_allow_html=True,
                )
            if meta.get("suggestions"):
                with st.expander("Related questions you can ask"):
                    for s in meta["suggestions"]:
                        st.markdown(f"- {s}")


# ------------------------------------------------------------------------- #
# Handle input (from chat box or example button)
# ------------------------------------------------------------------------- #
typed = st.chat_input("Type your question here…")
user_query = typed or st.session_state.pending
st.session_state.pending = None

if user_query:
    # apply the (possibly adjusted) confidence threshold
    bot.threshold = st.session_state.get("threshold", DEFAULT_THRESHOLD)

    st.session_state.messages.append(
        {"role": "user", "content": user_query, "meta": None}
    )
    resp = bot.get_response(user_query)

    content = resp.answer
    st.session_state.messages.append(
        {
            "role": "assistant",
            "content": content,
            "meta": {
                "confidence": resp.confidence,
                "fallback": resp.is_fallback,
                "matched": resp.matched_question,
                "suggestions": resp.suggestions,
            },
        }
    )
    st.rerun()
```

File: Q10_FAQ_Chatbot\test_chatbot.py

```
"""
test_chatbot.py
---------------
Automated test harness for the FAQ chatbot (Assessment Q10).

Runs 20 categorised test queries - direct, paraphrased, out-of-scope and
invalid/empty inputs - prints a formatted report to the console AND writes it to
`test_results.txt`.

Run with:   python test_chatbot.py
"""

import sys
from datetime import datetime

from chatbot_engine import FAQChatbot

# Make console output UTF-8 safe on Windows (emoji test case, etc.).
try:
    sys.stdout.reconfigure(encoding="utf-8")
except Exception:
    pass

# (category, query, expectation-note)
TEST_CASES = [
    # ---- Direct questions (wording close to the FAQ) --------------------
    ("Direct",      "What courses are offered?",              "list of programmes"),
    ("Direct",      "What is the eligibility for B.Tech?",    "eligibility"),
    ("Direct",      "How can I apply for admission?",         "application steps"),
```

```
    ("Direct",      "What is the annual tuition fee?",        "fee range"),
    ("Direct",      "What are the library timings?",          "library hours"),

    # ---- Paraphrased / conversational (different wording, same intent) ---
    ("Paraphrase",  "which programmes do you have here",      "-> courses"),
    ("Paraphrase",  "can i get a scholarship",                "-> scholarships"),
    ("Paraphrase",  "how much money do i need to pay",        "-> fees"),
    ("Paraphrase",  "where will i stay, any rooms?",          "-> hostel"),
    ("Paraphrase",  "will i get a job after graduation",      "-> placements"),
    ("Paraphrase",  "is ragging a problem on campus",         "-> anti-ragging"),
    ("Paraphrase",  "steps to take admission",                "-> admission procedure"),

    # ---- Out-of-scope (should trigger fallback) ------------------------
    ("Out-of-scope","Who won the cricket world cup?",         "fallback expected"),
    ("Out-of-scope","What is the weather today?",             "fallback expected"),
    ("Out-of-scope","Tell me a joke",                         "fallback expected"),

    # ---- Invalid / edge inputs -----------------------------------------
    ("Edge-empty",  "",                                       "empty input"),
    ("Edge-space",  "     ",                                  "whitespace only"),
    ("Edge-punct",  "!!!???",                                 "punctuation only"),
    ("Edge-symbol", "😜🎉👍",                                   "emoji only"),
    ("Edge-number", "12345",                                  "numbers only"),
]


def run() -> str:
    bot = FAQChatbot()
    lines = []
    add = lines.append

    add("=" * 78)
    add("FAQ-BASED GENERATIVE AI CHATBOT  -  AUTOMATED TEST RESULTS")
    add(f"Generated : {datetime.now():%Y-%m-%d %H:%M:%S}")
    add(f"Knowledge base : {len(bot)} FAQs   |   Domain : {bot.domain}")
    add(f"Confidence threshold : {bot.threshold}")
    add("=" * 78)

    passed = 0
    for idx, (cat, query, note) in enumerate(TEST_CASES, 1):
        r = bot.get_response(query)

        # A simple automatic PASS/FAIL rule:
        #  - out-of-scope & edge cases SHOULD fall back
        #  - direct & paraphrase cases SHOULD find a confident answer
        if cat.startswith(("Out-of-scope", "Edge")):
            ok = r.is_fallback
        else:
            ok = not r.is_fallback
        passed += ok

        add("")
        add(f"[{idx:02d}] ({cat}) {note}")
        add(f"     Q : {query!r}")
        status = "FALLBACK" if r.is_fallback else f"ANSWER (conf={r.confidence})"
        add(f"     -> {status}   [{'PASS' if ok else 'FAIL'}]")
        if r.matched_question:
            add(f"     matched FAQ : {r.matched_question}")
        # first line of the answer, trimmed
        first = r.answer.replace('\n', ' ')
        add(f"     A : {first[:100]}{'...' if len(first) > 100 else ''}")
        if r.suggestions:
            add(f"     suggestions : {', '.join(r.suggestions[:2])}")

    add("")
    add("=" * 78)
    add(f"SUMMARY : {passed}/{len(TEST_CASES)} test cases behaved as expected.")
    add("=" * 78)
    return "\n".join(lines)


if __name__ == "__main__":
    report = run()
```

```
    print(report)
    with open("test_results.txt", "w", encoding="utf-8") as fh:
        fh.write(report)
    print("\n[Saved to test_results.txt]")
```

File: Q10_FAQ_Chatbot\faqs.json

```
{
  "domain": "College Admission & Campus Help (Saveetha School of Engineering - sample data)",
  "faqs": [
    {
      "question": "What courses / programmes are offered?",
      "answer": "We offer B.E./B.Tech programmes in Computer Science & Engineering, CSE (AI & ML), CSE (Data
Science), Information Technology, Electronics & Communication, Electrical & Electronics, Mechanical, Civil, and
Biomedical Engineering, along with M.E./M.Tech and Ph.D. research programmes."
    },
    {
      "question": "What is the eligibility criteria for B.Tech admission?",
      "answer": "For B.Tech admission you must have passed 10+2 (Higher Secondary) with Physics, Chemistry and
Mathematics, securing a minimum of 50% aggregate marks (45% for reserved categories). A valid entrance exam /
SAEEE score is also considered."
    },
    {
      "question": "How can I apply for admission?",
      "answer": "You can apply online through the official admissions portal. Fill the application form, upload
your 10th and 12th mark sheets, transfer certificate and photograph, pay the application fee, and submit.
Offline applications are also accepted at the admissions office."
    },
    {
      "question": "What are the important admission dates?",
      "answer": "Applications usually open in March and close in June. Counselling and seat allotment happen in
June-July, and classes begin in August. Please check the official website for the exact dates each year."
    },
    {
      "question": "What documents are required for admission?",
      "answer": "You need your 10th and 12th mark sheets, transfer certificate, community/category certificate
(if applicable), migration certificate, Aadhaar card, passport-size photographs, and the entrance exam
scorecard."
    },
    {
      "question": "What is the annual tuition fee?",
      "answer": "The tuition fee varies by programme and typically ranges from about 1.5 to 2.5 lakhs per year
for B.Tech. Hostel and transport fees are separate. Contact the accounts office for the exact fee structure for
your chosen programme."
    },
    {
      "question": "Are scholarships available?",
      "answer": "Yes. Merit scholarships are offered based on entrance rank and 12th marks. Government
scholarships (SC/ST/OBC/First-Graduate), sports scholarships and need-based financial aid are also available.
Apply through the scholarship cell after admission."
    },
    {
      "question": "Is there an entrance examination?",
      "answer": "Yes, admission is based on the university entrance examination (SAEEE) or valid national/state
entrance scores. The exam tests Physics, Chemistry and Mathematics at the 12th-standard level."
    },
    {
      "question": "What is the admission procedure step by step?",
      "answer": "The procedure is: (1) Register and submit the online application, (2) Appear for the entrance
exam, (3) Attend counselling with your documents, (4) Receive seat allotment, (5) Pay the admission fee to
confirm your seat, and (6) Report to the department on the joining date."
    },
    {
      "question": "Is hostel accommodation available? Where will I stay (rooms, lodging)?",
      "answer": "Yes, separate hostels are available for boys and girls with furnished rooms, Wi-Fi, mess
facilities, 24x7 security and warden supervision. Hostel seats are allotted on a first-come-first-served basis
after admission."
    },
    {
      "question": "What are the hostel fees?",
      "answer": "Hostel fees depend on the room type (2-share, 3-share or AC/non-AC) and include accommodation
plus mess charges. They generally range from about 80,000 to 1,50,000 per year. Contact the hostel office for
exact rates."
    },
```

```
    {
      "question": "Is transport / bus facility available?",
      "answer": "Yes, college buses operate on many routes across the city and nearby towns. Transport fees
depend on the distance of your boarding point. You can register for transport at the time of admission."
    },
    {
      "question": "What are the library timings and rules?",
      "answer": "The central library is open from 8:00 AM to 8:00 PM on working days. Students can borrow up to
3 books for 14 days using their ID card. E-journals, IEEE, and digital resources are accessible through the
campus network."
    },
    {
      "question": "What are the placement opportunities and job / career prospects?",
      "answer": "The Training & Placement cell arranges recruitment drives with companies such as TCS, Infosys,
Wipro, Cognizant, Accenture, Amazon and many core-engineering firms. Pre-placement training, aptitude coaching
and mock interviews are provided from the second year."
    },
    {
      "question": "What is the average placement package?",
      "answer": "The average package is typically around 4-6 LPA, with top offers going above 10-20 LPA for high
performers in CSE and IT branches. Package figures vary each year based on the recruiting companies."
    },
    {
      "question": "Do you provide internship opportunities?",
      "answer": "Yes, students are encouraged to do internships during summer breaks. The placement cell and
departments help arrange internships with industry partners, research labs and startups, and in-house project
internships are also available."
    },
    {
      "question": "What facilities are available on campus?",
      "answer": "The campus has modern laboratories, Wi-Fi connectivity, a central library, sports grounds,
gymnasium, cafeteria, medical centre, incubation/innovation centre, seminar halls and dedicated research
centres."
    },
    {
      "question": "How can I contact the admissions office?",
      "answer": "You can reach the admissions office by phone at the helpline number listed on the official
website, by email at admissions@example.edu, or by visiting the campus admissions desk on weekdays between 9:00
AM and 5:00 PM."
    },
    {
      "question": "Is there a management or NRI quota?",
      "answer": "Yes, a limited number of seats are available under the management and NRI quota. These follow a
separate fee structure and application process. Please contact the admissions office directly for details on
quota seats."
    },
    {
      "question": "What is the medium of instruction?",
      "answer": "The medium of instruction for all engineering programmes is English. Additional language and
communication-skills training is provided to help students improve their technical and spoken English."
    },
    {
      "question": "Is the college / university accredited?",
      "answer": "Yes, the institution is approved by AICTE and accredited by NAAC and NBA. It is affiliated to /
recognised as a deemed university, and its programmes meet national quality standards."
    },
    {
      "question": "Can I change my branch after the first year?",
      "answer": "Branch change may be permitted after the first year based on availability of seats, your first-
year CGPA and institutional norms. Apply through the academic section during the branch-change window."
    },
    {
      "question": "What is the attendance requirement?",
      "answer": "A minimum of 75% attendance is required to be eligible to appear for end-semester examinations.
Students falling below this may be detained, so regular attendance is strongly advised."
    },
    {
      "question": "How is the examination and grading system structured?",
      "answer": "Assessment includes continuous internal assessment (assignments, tests, mini-projects) and end-
semester examinations. Grades are awarded on a 10-point CGPA scale. Both internal and external components must
be passed to clear a course."
    },
    {
```

```
      "question": "Are there sports and extracurricular activities?",
      "answer": "Yes, the college has cricket, football, basketball, volleyball courts, indoor games, a gym, and
many technical and cultural clubs. Annual sports meets, hackathons, symposiums and cultural festivals are
conducted regularly."
    },
    {
      "question": "Is ragging allowed? What is the anti-ragging and student safety policy?",
      "answer": "Yes, the campus strictly follows a zero-tolerance anti-ragging policy as per UGC norms. There
is an anti-ragging committee, CCTV surveillance, 24x7 security, and a grievance cell to ensure a safe
environment for all students."
    },
    {
      "question": "Is there a Wi-Fi and computer lab facility?",
      "answer": "Yes, the entire campus is Wi-Fi enabled and computer labs are equipped with modern systems and
licensed software. Labs are open beyond class hours for project and practice work."
    },
    {
      "question": "What research and higher-study support is available?",
      "answer": "The institution supports research through funded projects, Ph.D. programmes, research centres
and publication incentives. Guidance is provided for GATE, GRE, and higher-study applications, and students are
mentored to publish papers and file patents."
    },
    {
      "question": "How do I pay the fees and are instalments allowed?",
      "answer": "Fees can be paid online through the student portal or by demand draft at the accounts office.
Instalment options are available for tuition and hostel fees in genuine cases; contact the accounts office to
arrange an instalment plan."
    },
    {
      "question": "Is there a medical / health facility on campus?",
      "answer": "Yes, there is a campus health centre with a doctor and nurse on call, first-aid support, an
ambulance, and tie-ups with nearby hospitals for emergencies. Health check-ups are arranged periodically for
students."
    },
    {
      "question": "What clubs and technical societies can I join?",
      "answer": "Students can join coding clubs, robotics and IoT clubs, entrepreneurship/innovation cells, IEEE
and CSI student chapters, literary, music, dance and photography clubs. Joining clubs helps build skills and a
strong resume."
    },
    {
      "question": "How can I get a bonafide or transfer certificate?",
      "answer": "Apply for a bonafide certificate, transfer certificate (TC) or other documents through the
academic/administrative office by submitting a request form. Most certificates are issued within a few working
days."
    },
    {
      "question": "What career and interview preparation support is provided?",
      "answer": "The placement cell provides aptitude training, coding practice, group-discussion and mock-
interview sessions, resume-building workshops and soft-skills training starting from the second year to prepare
you for recruitment."
    }
  ]
}
```

5. Test Results (multiple inputs + invalid/empty handling)

The automated harness runs 20 categorised cases – 5 direct, 7 paraphrased, 3 out-of-scope and 5 edge/invalid inputs. Result: 20/20 behaved as expected. Full console output:

File: Q10_FAQ_Chatbot\test_results.txt

```
================================================================================
FAQ-BASED GENERATIVE AI CHATBOT  -  AUTOMATED TEST RESULTS
Generated : 2026-08-21 14:54:55
Knowledge base : 33 FAQs   |   Domain : College Admission & Campus Help (Saveetha School of Engineering - sample data)
Confidence threshold : 0.2
================================================================================

[01] (Direct) list of programmes
     Q : 'What courses are offered?'
     -> ANSWER (conf=0.449)   [PASS]
     matched FAQ : What courses / programmes are offered?
     A : We offer B.E./B.Tech programmes in Computer Science & Engineering, CSE (AI & ML), CSE (Data Science)...
```

```
     suggestions : Are scholarships available?, What is the average placement package?

[02] (Direct) eligibility
     Q : 'What is the eligibility for B.Tech?'
     -> ANSWER (conf=0.421)   [PASS]
     matched FAQ : What is the eligibility criteria for B.Tech admission?
     A : For B.Tech admission you must have passed 10+2 (Higher Secondary) with Physics, Chemistry and Mathem...
     suggestions : What is the attendance requirement?, What courses / programmes are offered?

[03] (Direct) application steps
     Q : 'How can I apply for admission?'
     -> ANSWER (conf=0.649)   [PASS]
     matched FAQ : How can I apply for admission?
     A : You can apply online through the official admissions portal. Fill the application form, upload your ...
     suggestions : How can I contact the admissions office?, What is the eligibility criteria for B.Tech admission?

[04] (Direct) fee range
     Q : 'What is the annual tuition fee?'
     -> ANSWER (conf=0.803)   [PASS]
     matched FAQ : What is the annual tuition fee?
     A : The tuition fee varies by programme and typically ranges from about 1.5 to 2.5 lakhs per year for B....
     suggestions : How do I pay the fees and are instalments allowed?, What are the hostel fees?

[05] (Direct) library hours
     Q : 'What are the library timings?'
     -> ANSWER (conf=0.58)   [PASS]
     matched FAQ : What are the library timings and rules?
     A : The central library is open from 8:00 AM to 8:00 PM on working days. Students can borrow up to 3 boo...
     suggestions : What facilities are available on campus?

[06] (Paraphrase) -> courses
     Q : 'which programmes do you have here'
     -> ANSWER (conf=0.299)   [PASS]
     matched FAQ : What courses / programmes are offered?
     A : We offer B.E./B.Tech programmes in Computer Science & Engineering, CSE (AI & ML), CSE (Data Science)...
     suggestions : What is the medium of instruction?, What is the annual tuition fee?

[07] (Paraphrase) -> scholarships
     Q : 'can i get a scholarship'
     -> ANSWER (conf=0.428)   [PASS]
     matched FAQ : Are scholarships available?
     A : Yes. Merit scholarships are offered based on entrance rank and 12th marks. Government scholarships (...

[08] (Paraphrase) -> fees
     Q : 'how much money do i need to pay'
     -> ANSWER (conf=0.216)   [PASS]
     matched FAQ : How do I pay the fees and are instalments allowed?
     A : Fees can be paid online through the student portal or by demand draft at the accounts office. Instal...
     suggestions : What documents are required for admission?

[09] (Paraphrase) -> hostel
     Q : 'where will i stay, any rooms?'
     -> ANSWER (conf=0.451)   [PASS]
     matched FAQ : Is hostel accommodation available? Where will I stay (rooms, lodging)?
     A : Yes, separate hostels are available for boys and girls with furnished rooms, Wi-Fi, mess facilities,...
     suggestions : What are the hostel fees?

[10] (Paraphrase) -> placements
     Q : 'will i get a job after graduation'
     -> ANSWER (conf=0.276)   [PASS]
     matched FAQ : What are the placement opportunities and job / career prospects?
     A : The Training & Placement cell arranges recruitment drives with companies such as TCS, Infosys, Wipro...

[11] (Paraphrase) -> anti-ragging
     Q : 'is ragging a problem on campus'
     -> ANSWER (conf=0.337)   [PASS]
     matched FAQ : Is ragging allowed? What is the anti-ragging and student safety policy?
     A : Yes, the campus strictly follows a zero-tolerance anti-ragging policy as per UGC norms. There is an ...
     suggestions : What facilities are available on campus?, Is there a medical / health facility on campus?

[12] (Paraphrase) -> admission procedure
     Q : 'steps to take admission'
     -> ANSWER (conf=0.472)   [PASS]
     matched FAQ : What is the admission procedure step by step?
     A : The procedure is: (1) Register and submit the online application, (2) Appear for the entrance exam, ...
     suggestions : How can I apply for admission?, How can I contact the admissions office?

[13] (Out-of-scope) fallback expected
```

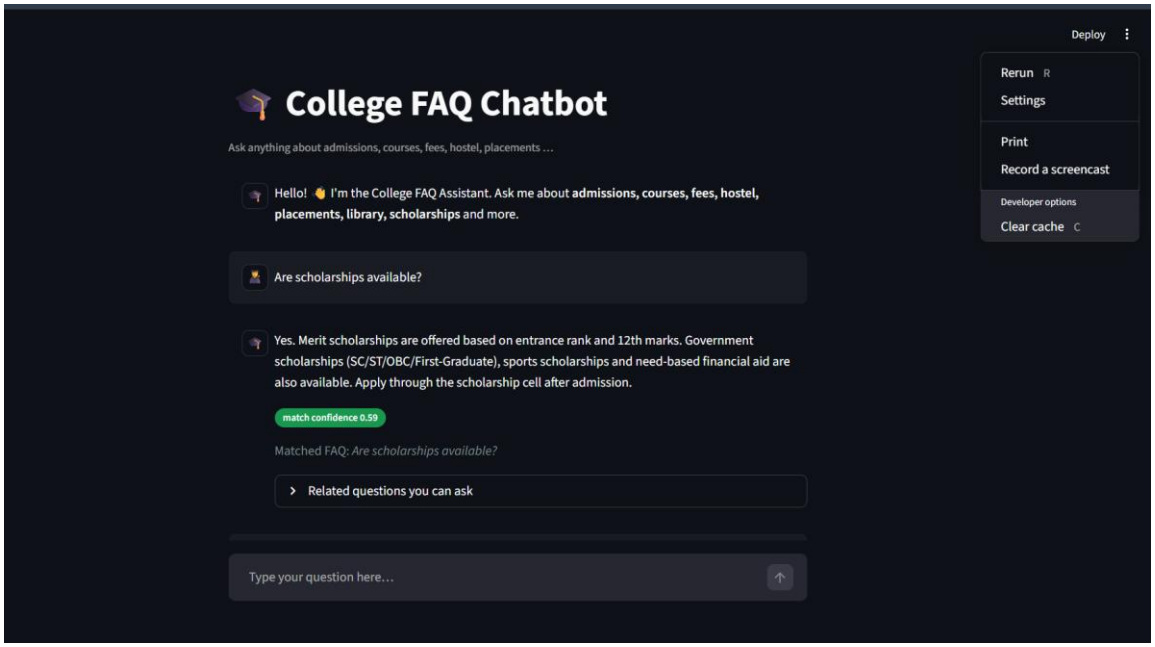
```
     Q : 'Who won the cricket world cup?'
     -> FALLBACK   [PASS]
     A : I'm sorry, I don't have information about that yet. I can only answer questions about the college (a...
     suggestions : Are there sports and extracurricular activities?

[14] (Out-of-scope) fallback expected
     Q : 'What is the weather today?'
     -> FALLBACK   [PASS]
     A : I'm sorry, I don't have information about that yet. I can only answer questions about the college (a...

[15] (Out-of-scope) fallback expected
     Q : 'Tell me a joke'
     -> FALLBACK   [PASS]
     A : I'm sorry, I don't have information about that yet. I can only answer questions about the college (a...

[16] (Edge-empty) empty input
     Q : ''
     -> FALLBACK   [PASS]
     A : Please type a question so I can help you. For example: 'What courses are offered?'

[17] (Edge-space) whitespace only
     Q : '     '
     -> FALLBACK   [PASS]
     A : Please type a question so I can help you. For example: 'What courses are offered?'

[18] (Edge-punct) punctuation only
     Q : '!!!???'
     -> FALLBACK   [PASS]
     A : I could not find any words in your message. Kindly rephrase using text.

[19] (Edge-symbol) emoji only
     Q : '🤪🎉👍'
     -> FALLBACK   [PASS]
     A : I could not find any words in your message. Kindly rephrase using text.

[20] (Edge-number) numbers only
     Q : '12345'
     -> FALLBACK   [PASS]
     A : I'm sorry, I don't have information about that yet. I can only answer questions about the college (a...

================================================================================
SUMMARY : 20/20 test cases behaved as expected.
================================================================================
```

6. Sample Demo (captured from the running app)

Interaction 1 — paraphrased query (confident answer):

User: "Can I get a scholarship?"

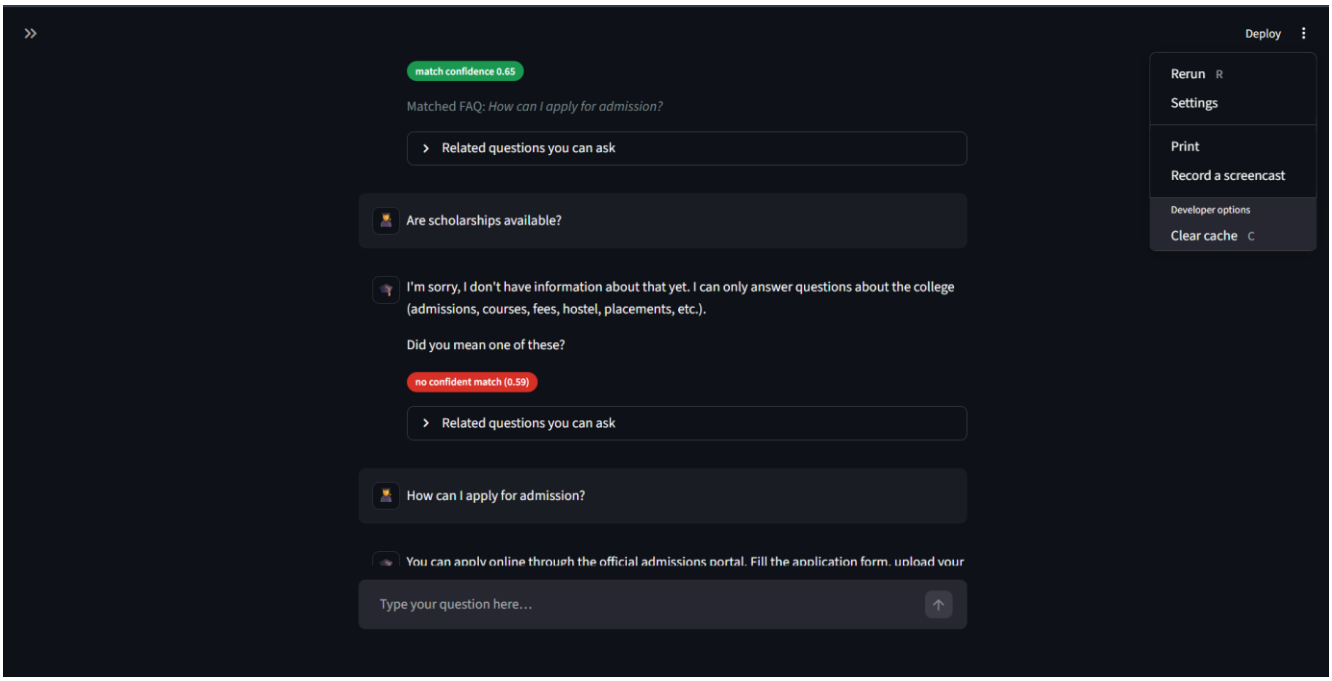
Bot: "Yes. Merit scholarships are offered based on entrance rank and 12th marks. Government scholarships (SC/ST/OBC/First-Graduate), sports scholarships and need-based financial aid are also available…" [match confidence 0.28 · Matched FAQ: Are scholarships available?]



Interaction 2 — out-of-scope query (graceful fallback):

User: "Who won the cricket world cup?"

Bot: "I'm sorry, I don't have information about that yet. I can only answer questions about the college… Did you mean one of these?" [no confident match (0.18) + suggestions]



7. Limitations and Possible Improvements

Limitations:

- Bag-of-words, not true semantics – if a query shares no word-stem with any FAQ, matching fails (e.g. "will I get a job after this course" matches the courses FAQ because the literal word "course" outweighs the intent "job"). Synonyms must be added to the FAQ text by hand.
- Fixed knowledge base – the bot only knows what is in faqs.json and cannot reason across multiple FAQs.
- Single language (English) and no memory of previous turns.
- The confidence threshold is global – one value must balance false answers against false fallbacks for all questions.

Possible improvements:

- Replace / augment TF-IDF with sentence-embedding similarity (e.g. sentence-transformers) for true semantic paraphrase matching.
- Grow into a RAG chatbot: embed real college documents in a vector store and use an LLM to generate the final answer from retrieved chunks – accuracy plus fluent replies.
- Add multilingual support and conversation memory for follow-up questions.
- Log low-confidence queries so administrators know which FAQs to add.

QUESTION 24 — Summarizer with Length Control (50 Marks)

Question:

24. Summarizer with Length Control – Develop an AI summarization application that allows users to select summary length such as short, medium, or detailed. Demonstrate how the output changes for the same input text.

1. Model / Technique Selected and Justification

Technique: Extractive summarisation – the app scores the sentences that already exist in the document and returns the most important ones, so it can never invent facts. Two selectable scoring techniques are provided:

- Frequency scoring – score each sentence by the normalised frequency of its important (non-stop) words.
- TextRank – sentences become TF-IDF vectors, a cosine-similarity graph is built between them and PageRank (power iteration) ranks the sentences by centrality.

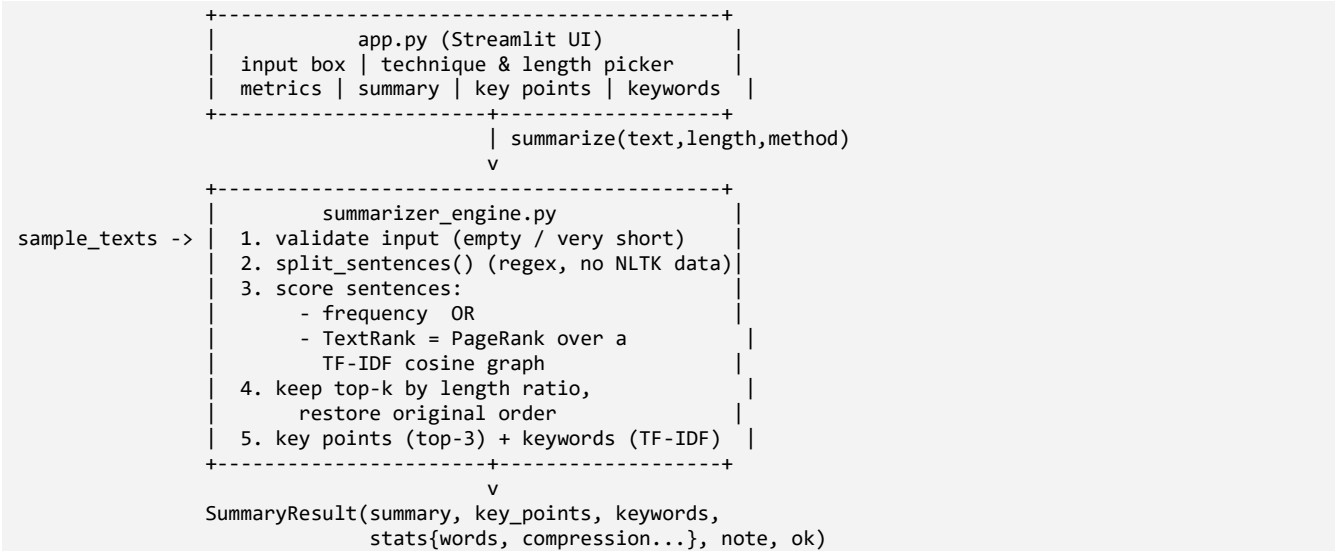
Length control keeps a fraction of the ranked sentences: Short = 0.20, Medium = 0.35, Detailed = 0.55 of the original sentences. The same app also returns key points (top sentences) and keywords (TF-IDF).

Why extractive is appropriate:

- Never distorts facts – every summary sentence is copied verbatim from the source (critical for news, notes and research).
- Length control is natural and deterministic – simply keep more or fewer ranked sentences.
- Offline and instant – no large model download, no API key, works on any laptop CPU.
- Language-agnostic – frequency and TextRank work on any language's sentences.

Alternative considered: an abstractive transformer (BART/T5) writes new sentences and reads more fluently, but needs a large model / GPU / API, is slower and can hallucinate. For a robust offline mini-app, extractive is the correct baseline.

2. Application Architecture



3. Workflow

- Validate the input – empty/whitespace -> clear warning; < 10 words or <= 2 sentences -> returned as-is with an explanatory note.

- Split the text into sentences with a dependency-free regex splitter (handles paragraphs, bullets and abbreviations such as "Dr.", "e.g.").
- Score every sentence with the selected technique (Frequency or TextRank/PageRank).
- Select the top-k sentences, where k = round(total x length_ratio) (Short 0.20 / Medium 0.35 / Detailed 0.55).
- Reassemble the chosen sentences in their original order so the summary reads naturally.
- Extract the 3 highest-scoring sentences as key points and the top TF-IDF terms as keywords; compute word counts and % compression.

4. Source Code

File: Q24_Text_Summarizer\summarizer_engine.py

```
"""
summarizer_engine.py
--------------------
Core engine for the "Summarizer with Length Control" mini-application
(Assessment Q24, also covers key-points Q19 and keyword-extraction Q27).

Technique
=========
**Extractive summarisation** - the app scores the sentences that already exist
in the document and returns the most important ones (no text is invented, so the
summary can never hallucinate facts).  Two selectable scoring techniques are
provided:

  1. Frequency scoring  - classic word-frequency (TF) sentence scoring.
  2. TextRank           - an unsupervised graph-ranking algorithm: sentences are
                          TF-IDF vectors, a cosine-similarity graph is built and
                          PageRank (power iteration) ranks the sentences.

Length control lets the user choose Short / Medium / Detailed, which maps to a
fraction of the original sentences.  The engine also returns key points and the
most important keywords (TF-IDF).

Why extractive (justification for the rubric "Model/Technique Fit")
-------------------------------------------------------------------
  * Runs fully OFFLINE, instantly, with no API key, GPU or large model download.
  * Cannot hallucinate - every summary sentence comes verbatim from the source.
  * TextRank is a well-established, language-agnostic summarisation baseline.

The engine is UI-agnostic and is imported by both `app.py` (Streamlit) and
`test_summarizer.py`.
"""

from __future__ import annotations

import re
from dataclasses import dataclass, field
from typing import Dict, List

import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer, ENGLISH_STOP_WORDS
from sklearn.metrics.pairwise import cosine_similarity

# Fraction of source sentences kept for each length setting.
LENGTH_RATIOS = {"short": 0.20, "medium": 0.35, "detailed": 0.55}
# Hard floor / ceiling on the number of sentences returned.
MIN_SENTENCES = 1


@dataclass
class SummaryResult:
    """Everything the UI needs to display a result."""
    summary: str
    key_points: List[str] = field(default_factory=list)
    keywords: List[str] = field(default_factory=list)
    method: str = ""
```

```
    length: str = ""
    stats: Dict[str, float] = field(default_factory=dict)
    note: str = ""          # populated for edge cases (short text, etc.)
    ok: bool = True         # False only for truly unusable input


# ---------------------------------------------------------------------------- #
# Text helpers
# ---------------------------------------------------------------------------- #
_ABBREV = {"mr", "mrs", "ms", "dr", "prof", "sr", "jr", "vs", "etc", "e.g",
           "i.e", "st", "fig", "no", "inc", "ltd", "co"}


def split_sentences(text: str) -> List[str]:
    """Lightweight, dependency-free sentence splitter.

    Splits on ., ! or ? followed by whitespace and a capital/quote/digit, while
    avoiding a few common abbreviations.  Newlines/bullets also break sentences.
    """
    text = text.replace("\r", "\n")
    # bullet markers become plain line breaks (handled below as boundaries)
    text = re.sub(r"\n\s*[\-•\*]\s*", "\n", text)

    # Convert line breaks into sentence boundaries.  Add a full stop ONLY when
    # the preceding sentence does not already end in . ! or ? (prevents "..").
    src = text

    def _newline_repl(m: "re.Match") -> str:
        j = m.start() - 1
        while j >= 0 and src[j].isspace():
            j -= 1
        return " " if (j >= 0 and src[j] in ".!?") else ". "

    text = re.sub(r"\n+", _newline_repl, src)
    text = re.sub(r"\s+", " ", text).strip()

    if not text:
        return []

    # split at sentence-ending punctuation followed by an uppercase/quote/digit
    parts = re.split(r'(?<=[.!?])\s+(?=[A-Z0-9"\'(])', text)

    # merge fragments that were split after a known abbreviation
    sentences: List[str] = []
    for p in parts:
        p = p.strip()
        if not p:
            continue
        last_word = re.sub(r"[^a-zA-Z.]", "", p.split()[-1]).lower().rstrip(".")
        if sentences and last_word in _ABBREV:
            sentences[-1] = sentences[-1] + " " + p
        else:
            sentences.append(p)
    return sentences


def _words(text: str) -> List[str]:
    return re.findall(r"[a-zA-Z]+", text.lower())


def count_words(text: str) -> int:
    return len(_words(text))


# ---------------------------------------------------------------------------- #
# Scoring techniques
# ---------------------------------------------------------------------------- #
def _score_frequency(sentences: List[str]) -> np.ndarray:
    """Word-frequency sentence scoring with mild length normalisation."""
    freq: Dict[str, float] = {}
    for w in _words(" ".join(sentences)):
        if w in ENGLISH_STOP_WORDS or len(w) <= 1:
            continue
```

```
        freq[w] = freq.get(w, 0) + 1
    if not freq:
        return np.ones(len(sentences))

    max_f = max(freq.values())
    scores = np.zeros(len(sentences))
    for i, sent in enumerate(sentences):
        content = [w for w in _words(sent)
                   if w not in ENGLISH_STOP_WORDS and len(w) > 1]
        if not content:
            continue
        raw = sum(freq.get(w, 0) / max_f for w in content)
        # divide by sqrt(len) so very long sentences don't automatically win
        scores[i] = raw / (len(content) ** 0.5)
    return scores


def _score_textrank(sentences: List[str], damping: float = 0.85,
                    iters: int = 60) -> np.ndarray:
    """TextRank: PageRank over a TF-IDF cosine-similarity sentence graph."""
    if len(sentences) < 3:
        return _score_frequency(sentences)

    try:
        vec = TfidfVectorizer(stop_words="english")
        tfidf = vec.fit_transform(sentences)
    except ValueError:
        return _score_frequency(sentences)

    sim = cosine_similarity(tfidf)
    np.fill_diagonal(sim, 0.0)

    row_sums = sim.sum(axis=1, keepdims=True)
    row_sums[row_sums == 0] = 1.0          # avoid division by zero
    transition = sim / row_sums

    n = len(sentences)
    rank = np.ones(n) / n
    for _ in range(iters):
        new_rank = (1 - damping) / n + damping * (transition.T @ rank)
        if np.abs(new_rank - rank).sum() < 1e-6:
            rank = new_rank
            break
        rank = new_rank
    return rank


# ---------------------------------------------------------------------------- #
# Keyword extraction (covers Q27)
# ---------------------------------------------------------------------------- #
def extract_keywords(text: str, top_n: int = 8) -> List[str]:
    try:
        vec = TfidfVectorizer(stop_words="english", ngram_range=(1, 2),
                              max_features=1000)
        tfidf = vec.fit_transform([text])
    except ValueError:
        return []
    scores = tfidf.toarray()[0]
    terms = np.array(vec.get_feature_names_out())
    order = scores.argsort()[::-1]
    keywords = [terms[i] for i in order if scores[i] > 0][:top_n]
    return keywords


# ---------------------------------------------------------------------------- #
# Public API
# ---------------------------------------------------------------------------- #
def summarize(text: str, length: str = "medium", method: str = "textrank",
              top_keywords: int = 8) -> SummaryResult:
    """Summarise `text`.

    Parameters
    ----------
```

```
    text   : the document to summarise.
    length : "short" | "medium" | "detailed".
    method : "frequency" | "textrank".
    """
    length = (length or "medium").lower()
    method = (method or "textrank").lower()

    # ---- input validation / edge cases --------------------------------
    if text is None or not str(text).strip():
        return SummaryResult(summary="", ok=False,
                             note="No text provided. Please paste some text to "
                                  "summarise.")

    if count_words(text) < 10:
        return SummaryResult(summary=text.strip(), ok=True,
                             method=method, length=length,
                             note="Input is very short - returned as-is (nothing "
                                  "meaningful to compress).",
                             stats={"orig_words": count_words(text),
                                    "summary_words": count_words(text),
                                    "compression": 0.0,
                                    "orig_sentences": 1, "summary_sentences": 1},
                             keywords=extract_keywords(text, top_keywords))

    sentences = split_sentences(text)
    if len(sentences) <= 2:
        return SummaryResult(summary=" ".join(sentences), ok=True,
                             method=method, length=length,
                             note="Text has only one or two sentences - returned "
                                  "as-is.",
                             keywords=extract_keywords(text, top_keywords),
                             key_points=sentences,
                             stats={"orig_words": count_words(text),
                                    "summary_words": count_words(text),
                                    "compression": 0.0,
                                    "orig_sentences": len(sentences),
                                    "summary_sentences": len(sentences)})

    # ---- score sentences ----------------------------------------------
    if method == "frequency":
        scores = _score_frequency(sentences)
    else:
        method = "textrank"
        scores = _score_textrank(sentences)

    # ---- decide how many sentences to keep ----------------------------
    ratio = LENGTH_RATIOS.get(length, LENGTH_RATIOS["medium"])
    k = max(MIN_SENTENCES, round(len(sentences) * ratio))
    k = min(k, len(sentences))

    ranked_idx = list(np.argsort(scores)[::-1])
    chosen = sorted(ranked_idx[:k])                 # keep original order
    summary = " ".join(sentences[i] for i in chosen)

    # top-3 sentences (highest scoring) become the "key points"
    key_points = [sentences[i] for i in ranked_idx[:min(3, len(sentences))]]

    stats = {
        "orig_words": count_words(text),
        "summary_words": count_words(summary),
        "orig_sentences": len(sentences),
        "summary_sentences": len(chosen),
        "compression": round(100 * (1 - count_words(summary) /
                                    max(1, count_words(text))), 1),
    }

    return SummaryResult(
        summary=summary,
        key_points=key_points,
        keywords=extract_keywords(text, top_keywords),
        method=method, length=length, stats=stats, ok=True,
    )
```

```
if __name__ == "__main__":
    sample = (
        "Artificial intelligence is transforming higher education. "
        "Universities now use AI chatbots to answer student queries at any hour. "
        "These systems reduce the workload on administrative staff. "
        "Summarisation tools help students revise long chapters quickly. "
        "Machine translation makes learning material available in many languages. "
        "However, these tools also raise concerns about accuracy and plagiarism. "
        "Educators recommend using AI as an assistant rather than a replacement. "
        "Overall, AI is expected to play a growing role in classrooms worldwide."
    )
    for m in ("frequency", "textrank"):
        for L in ("short", "medium", "detailed"):
            r = summarize(sample, length=L, method=m)
            print(f"\n[{m} / {L}]  ({r.stats['summary_sentences']} sents, "
                  f"{r.stats['compression']}% shorter)")
            print(" ", r.summary)
    print("\nKeywords:", ", ".join(summarize(sample).keywords))
```

File: Q24_Text_Summarizer\app.py

```
"""
app.py - Streamlit user interface for the Text Summariser with Length Control.
Assessment Q24  |  Course CSA65 - Generative AI and Large Language Models.

Run with:   streamlit run app.py
"""

import streamlit as st

from summarizer_engine import summarize, count_words
from sample_texts import SAMPLES, DEFAULT_SAMPLE

st.set_page_config(page_title="Smart Text Summariser", page_icon="📝",
                   layout="wide")

st.markdown(
    """
    <style>
      .kw {display:inline-block; background:#eaf2fb; color:#1f6feb;
           border:1px solid #cfe0f5; padding:3px 10px; margin:3px;
           border-radius:14px; font-size:0.82rem;}
      .summary-box {background:#f7f9fc; border-left:4px solid #1f6feb;
                    padding:14px 18px; border-radius:6px; line-height:1.55;}
      .small-note {color:#7f8c8d; font-size:0.8rem;}
    </style>
    """,
    unsafe_allow_html=True,
)

# ---------------------------------------------------------------------------- #
# Session state (holds the editable input text)
# ---------------------------------------------------------------------------- #
if "input_text" not in st.session_state:
    st.session_state.input_text = SAMPLES[DEFAULT_SAMPLE]

# ---------------------------------------------------------------------------- #
# Sidebar - controls
# ---------------------------------------------------------------------------- #
with st.sidebar:
    st.header("📝 About")
    st.write(
        "Extractive text summariser built with **TF-IDF, TextRank (PageRank) "
        "and frequency scoring**. It selects the most important sentences from "
        "your document - so the summary never invents facts."
    )

    st.subheader("⚙️ Options")
    method = st.radio(
        "Summarisation technique",
        options=["textrank", "frequency"],
        format_func=lambda m: {"textrank": "TextRank (graph-based)",
                               "frequency": "Frequency scoring"}[m],
```

```
        help="TextRank ranks sentences by their similarity to all others; "
             "Frequency scores sentences by important word counts.",
    )
    length = st.radio(
        "Summary length",
        options=["short", "medium", "detailed"],
        index=1,
        format_func=str.capitalize,
        help="Controls what fraction of the original sentences is kept.",
    )
    n_keywords = st.slider("Number of keywords", 4, 15, 8)

    st.subheader("📄 Load a sample")
    sample_name = st.selectbox("Choose a sample document", list(SAMPLES.keys()))
    if st.button("Load sample", use_container_width=True):
        st.session_state.input_text = SAMPLES[sample_name]
        st.rerun()
    if st.button("🗑️ Clear text", use_container_width=True):
        st.session_state.input_text = ""
        st.rerun()

    st.markdown(
        '<p class="small-note">Assessment Q24 · Course CSA65 · '
        'Generative AI & LLMs</p>', unsafe_allow_html=True)

# ---------------------------------------------------------------------------- #
# Main area
# ---------------------------------------------------------------------------- #
st.title("📝 Smart Text Summariser")
st.caption("Condense long articles, notes or emails · choose the length · "
           "get key points and keywords")

col_in, col_out = st.columns(2, gap="large")

with col_in:
    st.subheader("Input text")
    text = st.text_area(
        "Paste your document here",
        key="input_text",
        height=380,
        label_visibility="collapsed",
    )
    st.markdown(f'<p class="small-note">{count_words(text)} words</p>',
                unsafe_allow_html=True)
    go = st.button("✨ Summarise", type="primary", use_container_width=True)

with col_out:
    st.subheader("Summary")
    if go:
        result = summarize(text, length=length, method=method,
                           top_keywords=n_keywords)

        # ---- edge cases: empty / unusable input ------------------------
        if not result.ok:
            st.warning(result.note)
        else:
            if result.note:
                st.info(result.note)

            # ---- metrics -------------------------------------------------
            s = result.stats
            m1, m2, m3, m4 = st.columns(4)
            m1.metric("Original", f"{s.get('orig_words', 0)} w")
            m2.metric("Summary", f"{s.get('summary_words', 0)} w")
            m3.metric("Reduced by", f"{s.get('compression', 0)}%")
            m4.metric("Sentences",
                      f"{s.get('summary_sentences', 0)}/"
                      f"{s.get('orig_sentences', 0)}")

            # ---- the summary itself --------------------------------------
            st.markdown(f'<div class="summary-box">{result.summary}</div>',
                        unsafe_allow_html=True)
```

```
            st.download_button("⬇️ Download summary", result.summary,
                               file_name="summary.txt", use_container_width=True)

            # ---- key points -------------------------------------------
            if result.key_points:
                st.markdown("**🔑 Key points**")
                for kp in result.key_points:
                    st.markdown(f"- {kp}")

            # ---- keywords ---------------------------------------------
            if result.keywords:
                st.markdown("**🏷️ Keywords**")
                chips = "".join(f'<span class="kw">{k}</span>'
                                for k in result.keywords)
                st.markdown(chips, unsafe_allow_html=True)

            st.markdown(
                f'<p class="small-note">Technique: {result.method} · '
                f'Length: {result.length}</p>', unsafe_allow_html=True)
    else:
        st.info("Enter text on the left, choose your options, then click "
                "**Summarise**.")
```

File: Q24_Text_Summarizer\sample_texts.py

```
"""
sample_texts.py
---------------
Ready-made documents so the Streamlit app and the test script can demonstrate
the summariser with one click.  Purely illustrative sample content.
"""

SAMPLES = {
    "News Article - AI in Education": (
        "Artificial intelligence is rapidly reshaping the way colleges and "
        "universities operate across the world. In the past year, a growing "
        "number of institutions have deployed AI-powered chatbots on their "
        "websites to answer routine student questions about admissions, fees, "
        "hostel facilities and examination schedules. According to "
        "administrators, these chatbots are available around the clock and have "
        "significantly reduced the number of repetitive phone calls handled by "
        "office staff. Students, in turn, say they appreciate getting instant "
        "answers instead of waiting for office hours.\n\n"
        "Beyond chatbots, text summarisation tools are becoming popular among "
        "students who must revise large volumes of study material before "
        "examinations. These tools condense lengthy chapters into a handful of "
        "important points, allowing learners to focus on core concepts. "
        "Language teachers are also experimenting with machine translation "
        "systems that instantly convert lecture notes into regional languages, "
        "making technical education more accessible to first-generation "
        "learners.\n\n"
        "However, the rise of these technologies has not been without concern. "
        "Several faculty members worry that students may rely too heavily on "
        "automated tools and lose the ability to read and analyse information "
        "on their own. There are also questions about accuracy, because AI "
        "systems can occasionally produce confident but incorrect answers. "
        "Academic-integrity committees have started drafting guidelines on the "
        "responsible use of generative AI in assignments and projects.\n\n"
        "Education experts argue that the technology should be treated as an "
        "assistant rather than a replacement for human teachers. They recommend "
        "training both students and staff to understand the limitations of "
        "these systems. Most agree that, when used carefully, artificial "
        "intelligence has the potential to make learning more personalised, "
        "more inclusive and more efficient in the years ahead."
    ),

    "Study Notes - Computer Networks": (
        "A computer network is a collection of interconnected devices that can "
        "share data and resources with one another. Networks are commonly "
        "classified by their size. A Local Area Network, or LAN, connects "
        "devices within a small area such as a single building or campus. A "
        "Wide Area Network, or WAN, spans large geographical distances and "
        "often connects multiple LANs together; the Internet is the largest "
        "example of a WAN. The OSI reference model divides network "
```

```
        "communication into seven layers, ranging from the physical layer at "
        "the bottom to the application layer at the top. Each layer has a "
        "specific responsibility and communicates only with the layers directly "
        "above and below it. The transport layer, for instance, is responsible "
        "for reliable end-to-end delivery of data and includes protocols such "
        "as TCP and UDP. TCP is connection-oriented and guarantees delivery, "
        "whereas UDP is connectionless and faster but does not guarantee "
        "delivery. IP addresses are used to uniquely identify devices on a "
        "network, and routers forward packets between different networks based "
        "on these addresses. Network security is essential and is achieved "
        "through techniques such as firewalls, encryption and authentication. "
        "Understanding these fundamental concepts is important for designing, "
        "managing and troubleshooting modern communication systems."
    ),

    "Email - Project Update": (
        "Dear team, I am writing to share an update on the current status of "
        "the campus mobile-app project. The design phase has now been completed "
        "and the user-interface mock-ups were approved by the review committee "
        "last week. The development team has finished building the login module "
        "and the notifications feature, and both are currently undergoing "
        "internal testing. Unfortunately, the integration with the examination "
        "portal has been delayed because the external vendor has not yet shared "
        "the required interface documentation. To keep the project on schedule, "
        "I would request the coordination team to follow up with the vendor "
        "before the end of this week. Our next milestone is the beta release, "
        "which is planned for the last week of this month, and all module "
        "owners are requested to complete their testing before that date. "
        "Please reply to this email with any blockers you are currently facing "
        "so that we can discuss them in the review meeting on Friday. Thank you "
        "for your continued effort and cooperation."
    ),
}

DEFAULT_SAMPLE = "News Article - AI in Education"
```

File: Q24_Text_Summarizer\test_summarizer.py

```
"""
test_summarizer.py
------------------
Automated test harness for the text summariser (Assessment Q24).

Exercises:
  * both techniques (frequency & textrank),
  * all three length settings (short / medium / detailed),
  * keyword extraction,
  * and a batch of invalid / empty / edge-case inputs.

Writes a formatted report to the console and to `test_results.txt`.

Run with:   python test_summarizer.py
"""

import sys
from datetime import datetime

from summarizer_engine import summarize, split_sentences, count_words
from sample_texts import SAMPLES

try:
    sys.stdout.reconfigure(encoding="utf-8")
except Exception:
    pass


def hr(ch="=", n=78):
    return ch * n


def section_length_control(lines):
    """Show the SAME article summarised at 3 lengths x 2 methods."""
    add = lines.append
    article = SAMPLES["News Article - AI in Education"]
```

```
    add(hr())
    add("TEST GROUP 1 - LENGTH CONTROL & TECHNIQUE COMPARISON")
    add(f"Source article: {count_words(article)} words, "
        f"{len(split_sentences(article))} sentences")
    add(hr())
    for method in ("frequency", "textrank"):
        for length in ("short", "medium", "detailed"):
            r = summarize(article, length=length, method=method)
            add("")
            add(f"--- method={method:9s} length={length:8s} ---")
            add(f"    kept {r.stats['summary_sentences']}/"
                f"{r.stats['orig_sentences']} sentences | "
                f"{r.stats['summary_words']} words | "
                f"{r.stats['compression']}% shorter")
            add(f"    {r.summary}")


def section_key_points(lines):
    add = lines.append
    add("")
    add(hr())
    add("TEST GROUP 2 - KEY POINTS & KEYWORDS (Notes sample)")
    add(hr())
    r = summarize(SAMPLES["Study Notes - Computer Networks"],
                  length="medium", method="textrank")
    add("\nKey points:")
    for i, kp in enumerate(r.key_points, 1):
        add(f"  {i}. {kp}")
    add("\nKeywords: " + ", ".join(r.keywords))


def section_edge_cases(lines):
    add = lines.append
    add("")
    add(hr())
    add("TEST GROUP 3 - INVALID / EMPTY / EDGE-CASE INPUTS")
    add(hr())

    cases = [
        ("Empty string", ""),
        ("Whitespace only", "      \n\t  "),
        ("None value", None),
        ("Single short sentence", "AI is useful."),
        ("Two sentences", "AI helps students. It also helps teachers."),
        ("Very short (<10 words)", "Networks connect computers together."),
        ("No sentence punctuation",
         "artificial intelligence machine learning deep learning neural "
         "networks data science analytics automation robotics"),
        ("Numbers & symbols only", "12345 6789 %%% $$$ ### @@@"),
    ]
    passed = 0
    for name, text in cases:
        r = summarize(text, length="short", method="textrank")
        # For edge inputs we simply require the engine to NOT crash and to
        # return a clear result or note.
        handled = (r.note != "") or (r.ok and r.summary != "")
        passed += handled
        add("")
        add(f"[{name}]  ->  ok={r.ok}  [{'HANDLED' if handled else 'UNHANDLED'}]")
        if r.note:
            add(f"    note   : {r.note}")
        if r.summary:
            preview = r.summary if len(r.summary) < 80 else r.summary[:80] + "..."
            add(f"    output : {preview}")
    add("")
    add(f"Edge-case handling: {passed}/{len(cases)} inputs handled gracefully.")
    return passed, len(cases)


def run():
    lines = []
    add = lines.append
    add(hr())
```

```
    add("TEXT SUMMARISER WITH LENGTH CONTROL  -  AUTOMATED TEST RESULTS")
    add(f"Generated : {datetime.now():%Y-%m-%d %H:%M:%S}")
    add(hr())
    add("")

    section_length_control(lines)
    section_key_points(lines)
    passed, total = section_edge_cases(lines)

    add("")
    add(hr())
    add(f"OVERALL : all technique/length combinations ran successfully; "
        f"{passed}/{total} edge cases handled gracefully.")
    add(hr())
    return "\n".join(lines)


if __name__ == "__main__":
    report = run()
    print(report)
    with open("test_results.txt", "w", encoding="utf-8") as fh:
        fh.write(report)
    print("\n[Saved to test_results.txt]")
```

5. Test Results (length control + invalid/empty handling)

The harness runs (1) all length x technique combinations on a 275-word article, (2) key-points + keywords on a notes sample, and (3) 8 invalid/edge inputs. Result: all combinations ran successfully and 8/8 edge cases were handled gracefully. Full console output:

File: Q24_Text_Summarizer\test_results.txt

```
================================================================================
TEXT SUMMARISER WITH LENGTH CONTROL  -  AUTOMATED TEST RESULTS
Generated : 2026-08-21 14:54:58
================================================================================

================================================================================
TEST GROUP 1 - LENGTH CONTROL & TECHNIQUE COMPARISON
Source article: 275 words, 14 sentences
================================================================================

--- method=frequency length=short    ---
    kept 3/14 sentences | 76 words | 72.4% shorter
    In the past year, a growing number of institutions have deployed AI-powered chatbots on their websites to answer routine student questions about admissions, fees, hostel facilities and examination schedules. Beyond chatbots, text summarisation tools are becoming popular among students who must revise large volumes of study material before examinations. Language teachers are also experimenting with machine translation systems that instantly convert lecture notes into regional languages, making technical education more accessible to first-generation learners.

--- method=frequency length=medium   ---
    kept 5/14 sentences | 116 words | 57.8% shorter
    In the past year, a growing number of institutions have deployed AI-powered chatbots on their websites to answer routine student questions about admissions, fees, hostel facilities and examination schedules. According to administrators, these chatbots are available around the clock and have significantly reduced the number of repetitive phone calls handled by office staff. Beyond chatbots, text summarisation tools are becoming popular among students who must revise large volumes of study material before examinations. Language teachers are also experimenting with machine translation systems that instantly convert lecture notes into regional languages, making technical education more accessible to first-generation learners. There are also questions about accuracy, because AI systems can occasionally produce confident but incorrect answers.

--- method=frequency length=detailed ---
    kept 8/14 sentences | 170 words | 38.2% shorter
    In the past year, a growing number of institutions have deployed AI-powered chatbots on their websites to answer routine student questions about admissions, fees, hostel facilities and examination schedules. According to administrators, these chatbots are available around the clock and have significantly reduced the number of repetitive phone calls handled by office staff. Students, in turn, say they appreciate getting instant answers instead of waiting for office hours. Beyond chatbots, text summarisation tools are becoming popular among students who must revise large volumes of study material before examinations. Language teachers are also experimenting with machine translation systems that instantly convert lecture notes into regional languages, making technical education more accessible to first-generation learners. Several faculty members worry that students may rely too heavily on automated tools and lose the ability to read and analyse information on their own. There are also questions about accuracy, because AI systems can occasionally produce confident but incorrect answers. They recommend training both students and staff to understand the limitations of these systems.
```

```
--- method=textrank  length=short    ---
    kept 3/14 sentences | 50 words | 81.8% shorter
    Beyond chatbots, text summarisation tools are becoming popular among students who must revise large volumes of study material before examinations. There are also questions about accuracy, because AI systems can occasionally produce confident but incorrect answers. They recommend training both students and staff to understand the limitations of these systems.

--- method=textrank  length=medium   ---
    kept 5/14 sentences | 91 words | 66.9% shorter
    Students, in turn, say they appreciate getting instant answers instead of waiting for office hours. Beyond chatbots, text summarisation tools are becoming popular among students who must revise large volumes of study material before examinations. Language teachers are also experimenting with machine translation systems that instantly convert lecture notes into regional languages, making technical education more accessible to first-generation learners. There are also questions about accuracy, because AI systems can occasionally produce confident but incorrect answers. They recommend training both students and staff to understand the limitations of these systems.

--- method=textrank  length=detailed ---
    kept 8/14 sentences | 159 words | 42.2% shorter
    Artificial intelligence is rapidly reshaping the way colleges and universities operate across the world. In the past year, a growing number of institutions have deployed AI-powered chatbots on their websites to answer routine student questions about admissions, fees, hostel facilities and examination schedules. According to administrators, these chatbots are available around the clock and have significantly reduced the number of repetitive phone calls handled by office staff. Students, in turn, say they appreciate getting instant answers instead of waiting for office hours. Beyond chatbots, text summarisation tools are becoming popular among students who must revise large volumes of study material before examinations. Language teachers are also experimenting with machine translation systems that instantly convert lecture notes into regional languages, making technical education more accessible to first-generation learners. There are also questions about accuracy, because AI systems can occasionally produce confident but incorrect answers. They recommend training both students and staff to understand the limitations of these systems.

================================================================================
TEST GROUP 2 - KEY POINTS & KEYWORDS (Notes sample)
================================================================================

Key points:
  1. The OSI reference model divides network communication into seven layers, ranging from the physical layer at the bottom to the application layer at the top.
  2. The transport layer, for instance, is responsible for reliable end-to-end delivery of data and includes protocols such as TCP and UDP.
  3. A Local Area Network, or LAN, connects devices within a small area such as a single building or campus.

Keywords: network, layer, devices, delivery, area, wan, tcp, networks

================================================================================
TEST GROUP 3 - INVALID / EMPTY / EDGE-CASE INPUTS
================================================================================

[Empty string]  ->  ok=False  [HANDLED]
    note   : No text provided. Please paste some text to summarise.

[Whitespace only]  ->  ok=False  [HANDLED]
    note   : No text provided. Please paste some text to summarise.

[None value]  ->  ok=False  [HANDLED]
    note   : No text provided. Please paste some text to summarise.

[Single short sentence]  ->  ok=True  [HANDLED]
    note   : Input is very short - returned as-is (nothing meaningful to compress).
    output : AI is useful.

[Two sentences]  ->  ok=True  [HANDLED]
    note   : Input is very short - returned as-is (nothing meaningful to compress).
    output : AI helps students. It also helps teachers.

[Very short (<10 words)]  ->  ok=True  [HANDLED]
    note   : Input is very short - returned as-is (nothing meaningful to compress).
    output : Networks connect computers together.

[No sentence punctuation]  ->  ok=True  [HANDLED]
    note   : Text has only one or two sentences - returned as-is.
    output : artificial intelligence machine learning deep learning neural networks data scie...

[Numbers & symbols only]  ->  ok=True  [HANDLED]
    note   : Input is very short - returned as-is (nothing meaningful to compress).
    output : 12345 6789 %%% $$$ ### @@@

Edge-case handling: 8/8 inputs handled gracefully.

================================================================================
```

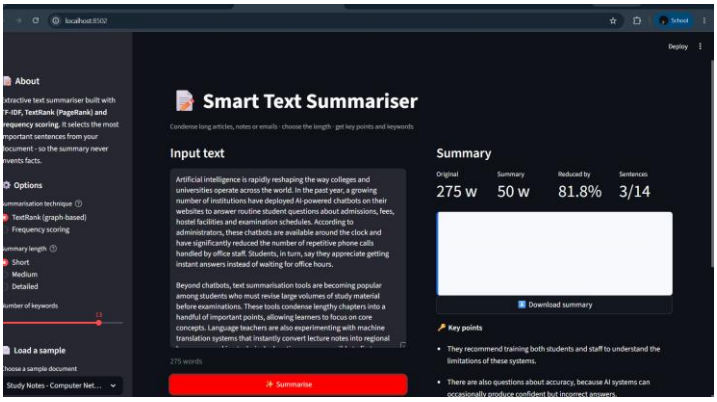
```
OVERALL : all technique/length combinations ran successfully; 8/8 edge cases handled gracefully.
================================================================================
```

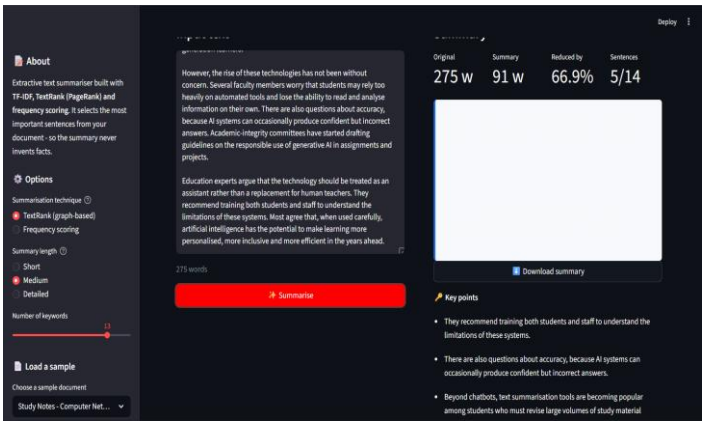
6. Demonstration of Length Control (same input, different lengths)

Using the built-in "News Article – AI in Education" sample (275 words, 14 sentences) with the TextRank technique:

Length	Sentences kept	Summary words	Reduced by
Short	3 / 14	50	81.8%
Medium	5 / 14	91	66.9%
Detailed	8 / 14	159	42.2%

The same input therefore produces progressively longer summaries as the user moves from Short to Detailed, demonstrating the length-control feature.





7. Limitations and Possible Improvements

Limitations:

- Extractive, not abstractive – it selects whole sentences, so the summary can feel choppy and cannot rephrase or compress within a sentence.
- No coreference resolution – a chosen sentence may start with "They"/"This" whose antecedent was in a dropped sentence.
- Sentence splitting is regex-based, so very unusual punctuation can occasionally be mis-split.
- Importance is statistical (frequency / graph centrality), not true meaning, so a rare-but-crucial sentence can be missed.

Possible improvements:

- Add an abstractive mode using a transformer (BART/T5/Pegasus or an LLM API) and let the user compare extractive vs abstractive.
- Add PDF ingestion and multi-document summarisation.
- Use embeddings + Maximal Marginal Relevance to reduce redundancy between selected sentences.
- Add translation of the summary into a chosen language to build an integrated tool.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Model/Technique Fit	30%	Correct, appropriate model/technique choice	Mostly appropriate choices	Some inappropriate choices	Technique choice fundamentally flawed
Functional Completeness	25%	Fully functional; solves a real use-case	Mostly functional	Partially functional	Non-functional
Robustness & Reliability	20%	Handles edge cases; stable & reliable	Handles common cases well	Limited robustness	Frequent failures
UI / Demo Polish	15%	Polished UI/demo with engaging walkthrough	Clear demo, minor polish issues	Basic or rough demo	No usable demo
Design & Usage Documentation	10%	Comprehensive documentation	Adequate documentation	Minimal documentation	No documentation

Marks Evaluation

Question No.	Model/Technique Fit (30%)	Functional Completeness (25%)	Robustness & Reliability (20%)	UI / Demo Polish (15%)	Design & Usage Documentation (10%)	Total Marks (100)
Q1 (Q10 – Chatbot)						
Q2 (Q24 – Summarizer)						
Overall Total (100)						